

pdf

https://github.com/SETAPESU26/05_balloon_pop this is my repo link guide me to this slab step by step such that i shld get full marks and also dot even miss easy steps tell me everything strp by step from start what to do and completely

Fixing the game engine to handle game-over states.

Fixing the game engine to handle game-over states.

Only game/game_engine.py needs to change for this task. balloon.py and renderer.py stay the same. The renderer already has a draw_banner function, which I'll reuse for the Game Over message.

Updated game/game_engine.py

python

"""

GameEngine: owns all balloons, spawns new ones, and handles clicks.

Version 3: three balloon types (normal, bonus, penalty) plus a lives system. The player starts with 3 lives and loses one whenever a balloon reaches the bottom without being popped. The game ends at 0 lives.

No timer yet.

"""

import random

from game.balloon import Balloon, BALLOON_TYPES

from game.click_detection import check_pop

from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45

STARTING_LIVES = 3

```

class GameEngine:

    def __init__(self):
        self.balloons = []
        self.frames_until_spawn = 0
        self.score = 0
        self.lives = STARTING_LIVES
        self.game_over = False

    def _choose_kind(self):
        kinds = list(BALLOON_TYPES.keys())
        weights = [BALLOON_TYPES[k]["weight"] for k in kinds]
        return random.choices(kinds, weights=weights, k=1)[0]

    def _spawn_balloon(self):
        radius = random.randint(16, 44)
        x = random.randint(radius + 10, WIDTH - radius - 10)
        speed = random.uniform(1.5, 3.0)
        kind = self._choose_kind()
        self.balloons.append(
            Balloon(x=x, y=-radius, radius=radius, speed=speed, kind=kind)
        )

    def handle_click(self, pos):
        # Ignore clicks once the game has ended.
        if self.game_over:
            return

```

```
popped = check_pop(self.balloons, pos)
```

```
if popped is not None:
```

```
    self.balloons.remove(popped)
```

```
    self.score += popped.points
```

```
    # Note: popping a balloon never changes self.lives.
```

```
def update(self):
```

```
    # Freeze the game once it has ended: no spawning, no movement.
```

```
    if self.game_over:
```

```
        return
```

```
    self.frames_until_spawn -= 1
```

```
    if self.frames_until_spawn <= 0:
```

```
        self._spawn_balloon()
```

```
        self.frames_until_spawn = SPAWN_INTERVAL_FRAMES
```

```
    for b in self.balloons:
```

```
        b.update()
```

```
    # Balloons that reached the bottom unpopped cost a life.
```

```
    missed = [b for b in self.balloons if b.is_past_bottom(HEIGHT)]
```

```
    if missed:
```

```
        self.lives -= len(missed)
```

```
        self.balloons = [b for b in self.balloons if b not in missed]
```

```
    if self.lives <= 0:
```

```
        self.lives = 0
```

```
self.game_over = True
```

```
def draw(self, surface, font):
```

```
    from game import renderer
```

```
    renderer.draw_scene(surface, self.balloons)
```

```
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
```

```
    renderer.draw_text(surface, font, f"Lives: {self.lives}", (WIDTH - 130, 10))
```

```
    if self.game_over:
```

```
        renderer.draw_banner(surface, font, f"Game Over! Final Score: {self.score}")
```

What changed and why

- **New constant STARTING_LIVES = 3**, so the starting value is easy to change later.
- **__init__** now sets self.lives and self.game_over.
- **update** now finds every balloon that has gone past the bottom (missed), subtracts one life for each, and then removes them from the list. Before, they were silently discarded. If lives hits 0, game_over becomes True.
- **The early return on game_over** in update stops new balloons from spawning and stops movement.
- **handle_click** ignores clicks after game over. The pop logic only touches score, never lives, so **popping can never cost a life**, whatever the type.
- **draw** shows Lives: N in the top-right corner. When the game is over, it draws the Game Over banner with the final score using the existing draw_banner.

Design notes

- **Any balloon that falls costs a life**, including penalty ones, because the README says "a balloon reaches the bottom without being popped." If you'd rather let penalty balloons fall for free, change missed to [b for b in self.balloons if b.is_past_bottom(HEIGHT) and b.kind != "penalty"] and filter them out of the list separately. The README's literal wording is safer for grading, so I kept it as is.
- I used self.lives = 0 inside the game-over check so the display never shows a negative number.
- There's no restart yet. That's Task 4, so for now you close the window to play again.

How to test

1. Save the file and run `python main.py`.
2. Watch the top-right corner. It should start at Lives: 3.
3. Let a balloon fall off the bottom. Lives should go to 2.
4. Pop red, gold, and dark gray balloons. Lives should **not** change.
5. Let three balloons fall. The "Game Over! Final Score" message should appear and the game should freeze. Clicking should do nothing.

I couldn't see `main.py`, so I assumed it calls `engine.update()`, `engine.handle_click(pos)`, and `engine.draw(screen, font)` like the starter code. If the banner looks off-center or clipped, tell me what you see and paste `main.py`.

Once it works, tell me and we'll move on to the timer and restart.

done with game over next whats next

"""

Balloon Pop (Lab Starter)

Run with: `python3 main.py`

Click balloons to pop them before they reach the bottom.

"""

```
import pygame
```

```
from game.game_engine import GameEngine
```

```
from game.renderer import WINDOW_SIZE
```

```
def main():
```

```
    pygame.init()
```

```
    screen = pygame.display.set_mode(WINDOW_SIZE)
```

```

pygame.display.set_caption("Balloon Pop")
clock = pygame.time.Clock()
font = pygame.font.SysFont("consolas", 22)

engine = GameEngine()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
        elif event.type == pygame.MOUSEBUTTONDOWN:
            engine.handle_click(event.pos)

    engine.update()
    engine.draw(screen, font)

    pygame.display.flip()
    clock.tick(60)

pygame.quit()

```

```

if __name__ == "__main__":
    main()

```

Designing the timer, game-over overlay, and restart handling.

Next is **Task 4: the 30-second timer, final score screen, and restart**. This one touches three files: `game_engine.py`, `renderer.py`, and `main.py`. `balloon.py` and `click_detection.py` stay the same.

Updated `game/game_engine.py`

python

"""

GameEngine: owns all balloons, spawns new ones, and handles clicks.

Version 4: three balloon types, a lives system, and a 30-second timed round. The round ends when time runs out OR lives reach zero. Press R after the round ends to start a new one.

"""

import random

import pygame

from game.balloon import Balloon, BALLOON_TYPES

from game.click_detection import check_pop

from game.renderer import WIDTH, HEIGHT

SPAWN_INTERVAL_FRAMES = 45

STARTING_LIVES = 3

ROUND_SECONDS = 30

class GameEngine:

def __init__(self):

self.reset()

def reset(self):

"""Start a fresh round: score, lives, timer, and balloons all reset."""

```

self.balloons = []

self.frames_until_spawn = 0

self.score = 0

self.lives = STARTING_LIVES

self.time_left = float(ROUND_SECONDS)

self.round_start = pygame.time.get_ticks()

self.game_over = False

self.end_reason = ""

def _end_round(self, reason):

    self.game_over = True

    self.end_reason = reason

def _choose_kind(self):

    kinds = list(BALLOON_TYPES.keys())

    weights = [BALLOON_TYPES[k]["weight"] for k in kinds]

    return random.choices(kinds, weights=weights, k=1)[0]

def _spawn_balloon(self):

    radius = random.randint(16, 44)

    x = random.randint(radius + 10, WIDTH - radius - 10)

    speed = random.uniform(1.5, 3.0)

    kind = self._choose_kind()

    self.balloons.append(

        Balloon(x=x, y=-radius, radius=radius, speed=speed, kind=kind)

    )

def handle_click(self, pos):

```

```

# No clicks are accepted once the round has ended.

if self.game_over:

    return


popped = check_pop(self.balloons, pos)

if popped is not None:

    self.balloons.remove(popped)

    self.score += popped.points

    # Popping a balloon never changes self.lives.


def handle_key(self, key):

    # Press R after the round ends to start a new one.

    if self.game_over and key == pygame.K_r:

        self.reset()


def update(self):

    # Frozen once the round has ended: no spawning, no movement.

    if self.game_over:

        return


    # Countdown timer (real time, so it is accurate even if FPS dips).

    elapsed = (pygame.time.get_ticks() - self.round_start) / 1000

    self.time_left = max(0.0, ROUND_SECONDS - elapsed)

    if self.time_left <= 0:

        self._end_round("Time's up!")

        return


self.frames_until_spawn -= 1

```

```

if self.frames_until_spawn <= 0:
    self._spawn_balloon()
    self.frames_until_spawn = SPAWN_INTERVAL_FRAMES

for b in self.balloons:
    b.update()

# Balloons that reached the bottom unpopped cost a life.
missed = [b for b in self.balloons if b.is_past_bottom(HEIGHT)]
if missed:
    self.lives -= len(missed)
    self.balloons = [b for b in self.balloons if b not in missed]

if self.lives <= 0:
    self.lives = 0
    self._end_round("Out of lives!")

def draw(self, surface, font):
    from game import renderer
    renderer.draw_scene(surface, self.balloons)
    renderer.draw_text(surface, font, f"Score: {self.score}", (10, 10))
    renderer.draw_text(surface, font, f"Time: {int(self.time_left + 0.999)}", (WIDTH // 2 - 50,
10))
    renderer.draw_text(surface, font, f"Lives: {self.lives}", (WIDTH - 130, 10))

if self.game_over:
    renderer.draw_game_over(surface, font, self.end_reason, self.score)

```

Updated game/renderer.py

python

"""

renderer: all pygame drawing lives here, kept separate from game logic.

"""

import pygame

WIDTH, HEIGHT = 700, 500

WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (200, 230, 245)

COLOR_TEXT = (30, 30, 30)

def draw_scene(surface, balloons):

surface.fill(COLOR_BG)

for b in balloons:

pygame.draw.circle(surface, b.color, (int(b.x), int(b.y)), b.radius)

pygame.draw.line(surface, (120, 120, 120), (b.x, b.y + b.radius), (b.x, b.y + b.radius + 12),
2)

def draw_text(surface, font, text, pos, color=COLOR_TEXT):

surface.blit(font.render(text, True, color), pos)

def draw_banner(surface, font, text):

surf = font.render(text, True, (180, 40, 40))

```
rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height() // 2))
surface.blit(surf, rect)
```

```
def draw_game_over(surface, font, reason, score):
    """Dim the screen and show why the round ended, the final score,
    and how to restart."""
    overlay = pygame.Surface(surface.get_size(), pygame.SRCALPHA)
    overlay.fill((0, 0, 0, 150))
    surface.blit(overlay, (0, 0))

    lines = [
        ("GAME OVER", (255, 255, 255)),
        (reason, (255, 120, 120)),
        (f"Final Score: {score}", (255, 210, 0)),
        ("Press R to play again", (255, 255, 255)),
    ]

    center_x = surface.get_width() // 2
    start_y = surface.get_height() // 2 - 60
    for i, (text, color) in enumerate(lines):
        surf = font.render(text, True, color)
        rect = surf.get_rect(center=(center_x, start_y + i * 40))
        surface.blit(surf, rect)
```

Updated main.py

python

"""

Balloon Pop (Lab Starter)

Run with: `python3 main.py`

Click balloons to pop them before they reach the bottom.

Press R after the round ends to play again.

"""

```
import pygame
```

```
from game.game_engine import GameEngine
```

```
from game.renderer import WINDOW_SIZE
```

```
def main():
```

```
    pygame.init()
```

```
    screen = pygame.display.set_mode(WINDOW_SIZE)
```

```
    pygame.display.set_caption("Balloon Pop")
```

```
    clock = pygame.time.Clock()
```

```
    font = pygame.font.SysFont("consolas", 22)
```

```
    engine = GameEngine()
```

```
    running = True
```

```
    while running:
```

```
        for event in pygame.event.get():
```

```
            if event.type == pygame.QUIT:
```

```
                running = False
```

```
            elif event.type == pygame.MOUSEBUTTONDOWN:
```

```
                engine.handle_click(event.pos)
```

```
            elif event.type == pygame.KEYDOWN:
```

```
engine.handle_key(event.key)
```

```
engine.update()
```

```
engine.draw(screen, font)
```

```
pygame.display.flip()
```

```
clock.tick(60)
```

```
pygame.quit()
```

```
if __name__ == "__main__":
```

```
    main()
```

What changed and why

game_engine.py

- Added `ROUND_SECONDS = 30` and a `time_left` value. The timer uses `pygame.time.get_ticks()` (real time), so it stays accurate even if the frame rate dips.
- Moved all the starting values into a `reset()` method, and `__init__` now calls it. Restarting is just calling `reset()` again, which resets **score, lives, timer, and balloons** together.
- Added `_end_round(reason)`, which sets `game_over` and records why the round ended ("Time's up!" or "Out of lives!"). Whichever happens first ends the round.
- `handle_key` restarts the round when you press **R**, but only after the round has ended.
- Once `game_over` is set, `update` and `handle_click` both return early. That means **no spawning and no clicks are accepted**, as the README requires.

renderer.py

- Added `draw_game_over`, which dims the screen and shows "GAME OVER", the reason, the **final score**, and the restart instruction. I kept `draw_banner` in place, so nothing else breaks.

main.py

- Added a KEYDOWN branch that passes key presses to the engine so R works.

How to test

1. Save all three files and run python main.py.
2. The screen should show **Score** on the left, **Time** in the middle counting down from 30, and **Lives** on the right.
3. **Time-out case:** pop balloons until the timer hits 0. The overlay should say "Time's up!" with your final score, and clicking should do nothing.
4. **Lives case:** let three balloons fall. It should say "Out of lives!".
5. Press **R**. Score should return to 0, lives to 3, the timer to 30, and the screen should clear of balloons.

Tip for your AFTER video: temporarily set ROUND_SECONDS = 10 so the game-over screen and restart fit into a short recording. Change it back to 30 before you push, because the README requires a 30-second round.

What's left after this

1. Confirm all four tasks work, then record the **after.mp4** video.
2. Export the chat history as a PDF and copy the share link.
3. Create the Lab-4 folder with both videos, the code, the PDF, and the link.
4. git add ., git commit, and git push. Do not open a PR.

Once Task 4 works, tell me and I'll walk you through the final recording checklist and the push commands again